DWX DEVELOPER WORLD 2026 · 30. JUNI 2026

(.NET) Aspire in der Praxis

Von lokaler Entwicklung bis zum Deployment



Sven Hennessen
vensas GmbH

vensas

ÜBER MICH



Sven Hennessen

- 🛠️ Fast **20 Jahre** Software-Entwicklung & Architektur
- 🏢 Co-Gründer & Co-Geschäftsführer **vensas GmbH** (gegründet 2023)
- 🏭 Beratung und Entwicklung von **Unternehmenssoftware** für den Mittelstand
- 🚴 Wenn nicht am Rechner: Fahrrad fahren, Wandern oder Rennsimulator

AGENDA

Was erwartet euch heute?

1. Geschichte
2. Demo-Anwendungsfall
3. Aspire Core-Features
4. Aspire Deployment
5. Aspire & KI
6. Limits & Lessons Learned

HINTERGRUND

Von Tye zu .NET Aspire

19

Project Tye

Microsoft Labs
Experimentell

23

.NET Aspire Preview

Build 2023
Dev-Tooling-Fokus

24

Aspire GA

Mai 2024, .NET 8
Stabile API

25

Aspire eigenständig

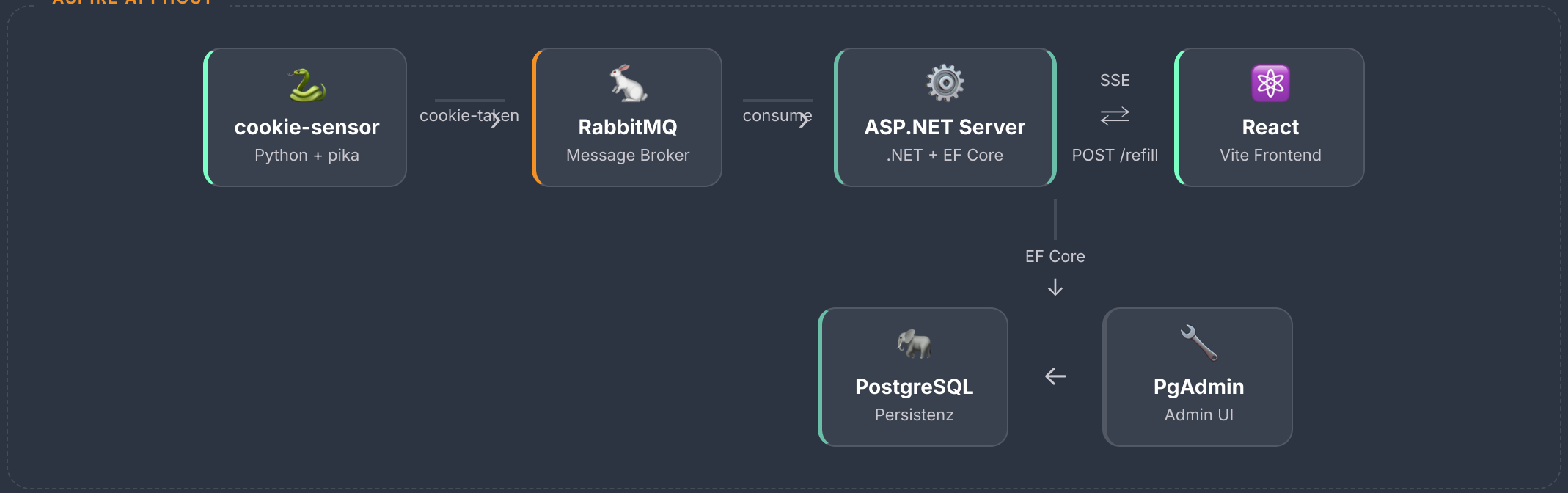
Kein .NET-Lock-in
eigene Versionierung

Heute · v 13.4

Aspire hat sich von einem reinen Dev-Tool zu einer ernstzunehmenden Plattform für verteilte Systeme entwickelt.

DEMO

Die Keksdose — Demo-Architektur



dotnet new aspire-ts-cs-starter

5 Ressourcen · 3 Sprachen · 1 AppHost

Aspire SDK 13.4.6

ASPIRE APPHOST

Lokale Orchestrierung

EIN KOMMANDO

```
aspire run
```

Alternativ: `dotnet run --project AppHost`

MEHRERE TECH-STACKS

.NET / C# via `AddProject<>`
Python via `AddPythonApp`
React / Vite via `AddViteApp`
Alles gleichwertig im AppHost.

WAITFOR & WITHREFERENCE

`.WaitFor(rabbitmq)` — wartet auf Healthy
`.WithReference(db)` — injiziert Umgebungsvariablen wie Connection Strings
Keine Race Conditions beim Startup. Built-in Service Discovery.

DASHBOARD

Strukturierte Logs, Traces & Metrics live im Browser.
OpenTelemetry out of the box
Services können **mehrere Endpoints** exponieren — Api, OpenApi-Dokumentation, etc.

KONFIGURATION

Parameter & Secrets

🔐 SECRET — KEIN DEFAULT

```
var pwd = builder.AddParameter(
  "rabbitmq-password",
  secret: true);
```

→ Gespeichert in `dotnet user-secrets`
→ Kein Wert = Fehler beim Start
→ Production: Azure Key Vault, Kubernetes Secrets

⚙️ PARAMETER — MIT DEFAULT

```
var cap = builder.AddParameter(
  "cookie-jar-capacity",
  "100"); // Default im Code
```

→ Kein Secret, kein user-secrets nötig
→ Injiziert als `CookieJar__Capacity`
→ `IOptionsMonitor<T>` im Service

```
# Lokal setzen
dotnet user-secrets set \
  "Parameters:rabbitmq-password" \
  "supersecret"
```

Beide Typen fließen als **Umgebungsvariablen** in die Services

UNTER DER HAUBE

Wie Aspire intern funktioniert

🖥️ **Dashboard** — läuft im AppHost-Prozess, kein separater Service

Services → OTLP — Logs, Traces, Metrics
Services ⇄ gRPC (DCP) — Status, Console-Output, Commands AppHost

🧩 Annotations-Modell

Alles ist ein `IResource` mit **Annotations** (`ContainerImageAnnotation`, `WaitForAnnotation` …).
`Build()` materialisiert das Modell.

🔗 WithReference

`.WithReference(db)` setzt beim Start `ConnectionStrings__<name>` als Env Var — dynamische Ports

🔍 Service Discovery

Aspire setzt `services__server__http__0=http://localhost:PORT`.
`AddServiceDefaults()` registriert einen Resolver — `http://server` funktioniert ohne Konfiguration.

❤️ WaitFor

State Machine: *Starting* → *Running* → *Healthy*.
`.WaitFor(x)` blockiert bis Healthy

DEPLOYMENT · TEIL 1

Azure & Docker Compose

☁️ Azure — First-Class Citizen

nicht Teil der Demo

```
# Azure Developer CLI – ein Befehl reicht:
azd init  # azure.yaml + Bicep generieren
azd up    # Infra + App provisionieren & deployen
```

- Aspire → Bicep: Container Apps, PostgreSQL Flexible Server, Service Bus
- `azd up` liest AppHost-Model
- Azure Key Vault für Secrets, Azure Monitor für Telemetrie
- Nicht alle Aspire-Ressourcen haben 1:1-Azure-Pendant

🐳 Docker Compose

Demo 1

```
// AppHost.cs
builder.AddDockerComposeEnvironment("docker-compose")
  .ConfigureComposeFile(compose => {
    compose.Services["messaging"].Healthcheck = …;
  });

# Artifacts generieren:
aspire publish --output-path ./deploy/output

# Starten:
docker compose -f ./deploy/output/docker-compose.yaml up
```

- Generiert: `docker-compose.yaml`
- `AddDockerfile` für non-.NET Container (nginx, Python, ...)
- `ConfigureComposeFile`: Healthchecks & Restart-Policies nachträglich ergänzen

DEPLOYMENT · TEIL 2

Kubernetes & Custom Providers

Kubernetes / Helm

Demo 2 · Preview

```
// AppHost.cs
builder.AddKubernetesEnvironment("kubernetes");

# Helm-Chart generieren:
aspire publish --output-path ./deploy/output

# Deployen:
helm upgrade --install cookiejar \
  ./deploy/output/kubernetes/
```

- Generiert Helm-Chart und `values.yaml`
- ⚠️ Preview-Feature
- Ausgabe ist Ausgangspunkt, kein produktionsfertiges Chart

Custom Provider — Kamal

Demo 3

```
// AppHost.cs – eigenen PipelineStep registrieren
builder.AddResource(new KamalDeploymentResource("kamal")
  { ServiceName = "cookiejar", Architecture = "arm64",
    ExtraSecretEnvVars = ["CookieJar__Capacity"] })
  .WithPipelineStepFactory(KamalPipelineStep.Create);

# Kamal-Artifacts generieren:
aspire publish --output-path ./deploy/output

# VM einrichten + deployen:
kamal deploy ...
```

- `PipelineStep` walkt das Resource-Model: liest DB-Namen, User, Health-Check-Pfad aus Annotationen
- Generiert `config/deploy.yml` + `.kamal/secrets.example`
- `RequiredBySteps` statt `RequiredBy(Publish)` — läuft nur explizit

ASPIRE MCP

KI-Tooling mit Anwendungskontext

```
# MCP-Server einrichten
aspire agent init

# → konfiguriert claude_desktop_config.json
# → oder andere MCP-fähige KI-Tools
```

Echter Mehrwert heute

- Schnelles Onboarding: KI versteht die Architektur sofort
- Live-Diagnose (Logs): "Warum schlägt Service X fehl?"
- Testing: KI kann Ressourcen direkt abfragen

Was der MCP-Server bereitstellt

- Laufende Ressourcen & deren Status
- Structured Logs live abrufbar
- Traces & Metrics
- Aspire Dokumentation durchsuchbar

👉 MCP macht Aspire zum **Kontext-Provider für Agenten** 🤖

LESSONS LEARNED

TL;DR

✅ Echter Mehrwert

- Lokale Entwicklung — massiver Zeitgewinn, gesamter Stack mit einem Befehl
- Beschreibungssprache näher am Entwickler
- Tech-Stack-Unabhängigkeit: Python, Node, Dockerfile — kein .NET-Zwang
- Debugging verteilter Systeme: WaitFor, Replicas, Observability out-of-the-box
- Aspire MCP: Produktionsbugs lokal reproduzieren, Logs vergleichen — Kontext für Agenten

⚠️ Deployment-Realität

- Docker Compose: verbessert sich, aber für echte Produktion noch Handarbeit
- Kubernetes: Preview — Helm-Output ist Ausgangspunkt, nicht Endprodukt
- Custom Provider (Kamal): machbar — aber Aufwand sollte den Nutzen rechtfertigen
- Außerhalb Azure gilt: Deployment nicht unterschätzen — Aspire allein reicht nicht

Fazit:

- Aspire ist für lokale Entwicklung (& Azure) klar empfehlenswert
- Für andere Plattformen: Augen offen — es wird besser, ist aber noch nicht da
- Custom Provider/Pipelines sind interessant für spezielle Szenarien

FRAGEN?

Danke!



Feedback?

vensas.de

kontakt@vensas.de

+49 (0)681 30984780

linkedin.com/company/vensas-gmbh

github.com/vensas

vensas